

Hoja informativa: Diseñar y desarrollar dashboards con Dash

```
# Importar librerías para trabajar con dashboards

import dash # la librería dash
import dash_core_components as dcc # controles de dash
import dash_html_components as html # elementos del dashboard de dash
import plotly.graph_objs as go # gráficos plotly
```

```
# definición del diseño

external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']
app = dash.Dash(__name__, external_stylesheets=external_stylesheets)
app.layout = html.Div(children=[
    # tu código para gráficos y controles
])
```

```
# Encabezado del dashboard
html.H1(children = 'Header'),
```

```
# Gráfico del dashboard

dcc.Graph(
    figure = {
        'data': [
            # el conjunto de elementos gráficos de plotly a mostrar en el gráfico
        ],
        'layout': # las propiedades visuales del gráfico
    }, # este parámetro se omite si se forma dinámicamente
    id = 'graph_id' # identificador del gráfico
),
```

```
# Mostrar el gráfico de una tabla

go.Table(header = {'values': ['<b>Año</b>',
                              '<b>País</b>',
                              '<b>% de población urbana</b>'],
              'fill_color': 'lightgrey', # el color de fondo de la celda
              'align': 'center'}, # alineación de texto
        cells = {'values': table.T.values})
```

```
# selector de rango de tiempo
dcc.DatePickerRange(
    start_date = urbanization['Year'].dt.date.min(), # fecha de inicio con valor predeterminado
    end_date = urbanization['Year'].dt.date.max(), # fecha de finalización con valor predeterminado
    display_format = 'YYYY', # formato de visualización de fecha y tiempo
    id = 'dt_selector', # el identificador de control unívoco
),
```

#Checkboxes (casillas de verificació

```
dcc.Checklist( options = [{'label': 'Africa', 'value': 'afr'},
                           {'label': 'Eurasia', 'value': 'eur'},
                           {'label': 'Australia', 'value': 'au'},
                           {'label': 'Americas', 'value': 'am'}], # matriz con opciones
    # etiqueta – el valor que ve el usuario
    # valor – valor técnico para el procesamiento interno
    value = ['afr', 'eur', 'au', 'am'], # el conjunto de valores
    id = 'continent_selector' ) # el identificador de control unívoco
```

lista desplegable

```
dcc.Dropdown( options = [{'label': 'Africa', 'value': 'afr'},
                           {'label': 'Eurasia', 'value': 'eur'},
                           {'label': 'Australia', 'value': 'au'},
                           {'label': 'Americas', 'value': 'am'}], # matriz con opciones
    # etiqueta – el valor que ve el usuario
    # valor – valor técnico para el procesamiento interno
    value = ['afr', 'eur', 'au', 'am'], # el conjunto de valores
    multi = True, # permite la selección de múltiples elementos de la lista
    id = 'continent_selector') # el identificador de control unívoco
```

botón de radio

```
dcc.RadioItems(options = [{'label': 'Dog', 'value': 'dog',
                           {'label': 'Cat', 'value': 'cat'},
                           {'label': 'Opossum', 'value': 'possum'}],
    # matriz con opciones
    # etiqueta – el valor que ve el usuario
    # valor – valor técnico para el procesamiento interno
    value = 'possum', # el conjunto de valores
    id = 'pet_selector') # el identificador de control unívoco
```

la lógica de un dashboard que depende de los controles

```
@app.callback(
    [Output('trig_func', 'figure')],
    ],
```

```

    [Input('mode_selector', 'value'),
    ])
def update_figures(selected_mode):
    # código para actualizar el gráfico

```

```

# un caso con múltiples inputs (entradas)

@app.callback(
    [Output('plot_1', 'figure'),
    Output('plot_2', 'figure')
    ],
    [Input('control_1', 'value'),
    Input('control_2', 'value'),
    Input('control_3', 'value')
    ])
def update_figures(control_1_value, control_2_value, control_3_value):
    # crear gráficos dinámicos
    return (
        { # plot_1 graph
            'data': plot_1_data,
            'layout': go.Layout(...)
        },
        { # plot_2 graph
            'data': plot_2_data,
            'layout': go.Layout(...)
        },
    )

```

```

# un elemento div

html.Div([
    html.Div([
        html.Label('I use 9 out of 12 available columns:'),
    ], className = 'nine columns'),
    html.Div([
        html.Label('I use 3 out of 12 available columns:'),
    ], className = 'three columns'),
], className = 'row'),

```

```

# establecer la altura del gráfico en 25% del ancho de página

dcc.Graph(
    style = {'height': '25vw'},
    id = 'sales_by_platform'
)

```

```

# añadir una línea vacía entre dos elementos del dashboard

```

```
html.Br()
```